

Strukture podataka i algoritmi

Složenost algoritama

dr Danijel Čabarkapa

Akademija strukovnih studija Šabac

Odsek za medicinske i poslovno-tehnološke studije

Sadržaj:

1. Timing & counting programa
2. Analiza složenosti algoritama
3. O - Θ - Ω notacije i klase složenosti algoritama

Analiza algoritama

1. Timing & Counting programa

Zašto algoritmi?

- Da možete da rešavate probleme na pravi način.
- Da postanete vrhunski programer.

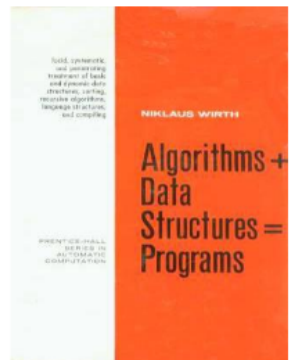
“Loš programer postaje dobar programer kada počne da brine i o kodu i o strukturi podataka. Loš programer brine samo o kodu. Dobar programer brine i o strukturi podataka i o njihovim relacijama.”

— Linus Torvalds (tvorac Linux-a)



“Algoritmi + Struktura podataka = Program”

- Niklaus Wirth (tvorac Pascal-a)



Efikasnost programa

- Računari su brzi i postaju sve brži – da li to znači da možda efikasni programi nisu važni?
 - Današnji skupovi podataka mogu biti veoma veliki (procena iz 2024. je da je ukupna količina skladištenih podataka kompanije Google u rasponu od 35–50 eksabajta ili desetine miliona terabajta, a indeksirano je na stotine milijardi veb stranica. **Koliko dugo traje pretraživanje grubom silom?**)
 - Jednostavan algoritam možda neće skalirati veličinu skupova podataka na prihvatljiv način
- Razdvajanje vremenske i prostorne efikasnosti programa
- Kompromis između prostora i vremena (*space-time tradeoff*): može se koristiti više memorije za čuvanje vrednosti i za brže pretraživanje kasnije
 - mogu li se rezultati prethodnih izračunavanja čuvati, a zatim koristiti „pretragu“ za brže dobijanje rezultata (npr. Fibonači niz sa čuvanjem pretrage)
- Fokusirati se na vremensku efikasnost algoritama

Efikasnost programa

- Izazovi u razumevanju efikasnosti rešenja računarskih problema:
 - program se može implementirati na mnogo različitih načina
 - problem se može rešiti koristeći samo nekoliko različitih algoritama
 - želi se razdvajanje izbora implementacije od izbora apstraktnijeg algoritma
- Kako izvršiti evaluaciju efikasnosti programa?
 - merenjem vremena izvršavanja (*timing*)
 - prebrojavanjem izvršenih operacija (*counting*)
 - apstraktni pojam redosleda porasta (*order of growth*)

najprikladniji način procene uticaja
izbora algoritma na rešavanje problema

Vreme izvršavanja programa

```
1  # Merenje vremena konverzije C u F
2
3  import time
4
5  def c_to_f(c):
6      return c*9.0/5 + 32
7
8  tstart= time.time()
9  c_to_f(50)
10 dt = time.time() -tstart
11 print(dt, "s,")
```

time modul

c_to_f(c) funkcija konverzije Celzijus u Farehnajt

Start clock

poziv funkcije

Stop clock

1.430511474609375e-06 s,

c_to_f(1) took 0.0 seconds
c_to_f(10) took 0.0 seconds
c_to_f(100) took 0.0 seconds
c_to_f(1000) took 0.0 seconds
c_to_f(10000) took 0.0 seconds
c_to_f(100000) took 0.0 seconds
c_to_f(1000000) took 0.0 seconds
c_to_f(10000000) took 0.0 seconds

Vremenska evaluacija programa

- **CILJ**: evaluacija različitih algoritama
- vreme izvršavanja varira između algoritama 
- vreme izvršavanja varira između implementacija 
- vreme izvršavanja varira između računara 
- vreme izvršavanja nije predvidljivo za male ulaze 

Vreme izvršavanja algoritma varira za različite opsege ulaza, ali ne može se formulisati stvarna vezu između njih



Broj operacija programa

- Pretpostavka je da konstantno vreme izvršavanja za: matematičke operacije, poređenja, dodele, pristup objektima u memoriji
- Prebrojati broj izvršenih operacija kao funkciju veličine ulaza

c_to_f $\rightarrow$ 3 ops

```
def c_to_f(c):  
    return c*9.0/5 + 32
```

3 ops

mysum $\rightarrow 1+(x+1)*(1+2) = 3x+4$ ops

```
def mysum(x):  
    total = 0  
    for i in range(x+1):  
        total += i  
    return total
```

1 op

loop
x+1 times

2 ops

1 op

square $\rightarrow 1+n*(1)*n*(1+2) = 3n^2 + 1$ ops

```
def square(n):  
    sqsum = 0  
    for i in range(n):  
        for j in range(n):  
            sqsum += 1  
    return sqsum
```

1 op

loop n times

loop n times

1 op

1 op

2 ops

Broj operacija programa

`c_to_f` → 3 ops

```
def c_to_f(c):  
    return c*9.0/5 + 32
```

3 ops



`c_to_f(100)`: 3 ops, 1.0 x more
`c_to_f(1000)`: 3 ops, 1.0 x more
`c_to_f(10000)`: 3 ops, 1.0 x more
`c_to_f(100000)`: 3 ops, 1.0 x more
`c_to_f(1000000)`: 3 ops, 1.0 x more
`c_to_f(10000000)`: 3 ops, 1.0 x more

Bez obzira na
veličinu ulaza, broj
operacija je ISTI

Broj operacija programa

$\text{mysum} \rightarrow 1+(x+1)*(1+2) = 3x+4$ ops

```
def mysum(x):
```

```
    total = 0
```

1 op

```
    for i in range(x+1):
```

loop
x+1 times

```
        total += i
```

1 op

```
    return total
```

2 ops

Ako se ulaz povećava
za 10, broj izvršenih
operacija je približno
10 puta veći



`mysum(100): 304 ops, 1.0 x more`

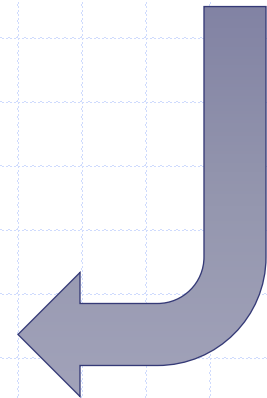
`mysum(1000): 3004 ops, 9.88158 x more`

`mysum(10000): 30004 ops, 9.98802 x more`

`mysum(100000): 300004 ops, 9.9988 x more`

`mysum(1000000): 3000004 ops, 9.99988 x more`

`mysum(10000000): 30000004 ops, 9.99999 x more`



Broj operacija programa

- **CILJ:** evaluacija različitih algoritama
- Broj operacija zavisi od algoritma 
- Broj operacija zavisi od implementacije 
- Broj operacija ne zavisi od računara 
- Nije precizno definisano koje operacije treba brojati 

Broj operacija algoritma varira za različite ulaze i može se pronaći veza između ulaza i broja operacija



Timing & Counting programa

- Merenje tačnog vremena izvršavanja programa:
 - Zavisi od računara
 - Zavisi od implementacije programa
 - Mali ulazi ne pokazuju porast vremena izvršavanja
- Prebrojavanje broja operacija:
 - Može se dobiti jednačinom!
 - Nezavisno je od računara
 - Zavisi od implementacije programa
 - Multiplikativne/aditivne konstante su irelevantne za velike ulaze
- Potrebno je:
 - Procena (evaluacija) algoritama
 - Evaluacija skalabilnosti
 - Evaluacija u smislu veličine ulaza

Analiza algoritama

2. Analiza složenosti algoritama

Složenost algoritama

- **Složenost algoritama** je vrlo važna tema u računarstvu koja se bavi analizom koliko resursa (vremena i memorije itd.) algoritam troši u odnosu na veličinu ulaznih podataka
- Ova vrsta analize omogućava da se razume koliko je algoritam efikasan, posebno kada obrađuje velike skupove podataka
- Ovaj koncept je kritičan za optimizaciju i sprečavanje problema sa performansama, posebno u velikim i složenim softverskim projektima
- Algoritmi visoke složenosti mogu dovesti do ozbiljnih problema sa performansama kada se koriste za velike skupove podataka
- Izbor algoritma i njegova optimizacija je pitanje koje se mora stalno razmatrati u procesu razvoja softvera

Složenost algoritama

- Cilj analize složenosti je da utvrdi ponašanje algoritma za različite kombinacije ulaznih veličina kako bi se omogućilo poređenje sa drugim rešenjima, u smislu utroška vremenskih i operativnih resursa
- Pored svojstva ispravnosti programa, veoma je važno i pitanje koliko program zahteva vremena (ili izvršenih instrukcija) i memorije za svoje izvršavanje
- Mnogi problemi mogu se rešavati na puno načina i nije svako rešenje jednako dobro
- Najbolja rešenja podrazumevaju minimalnu potrošnju resursa, pa se uglavnom razmatraju:
 - 1) vremenska složenost algoritma**
 - 2) prostorna (memorijska) složenost algoritama**

Procena vremena izvršavanja programa

- Vreme izvršavanja pojedinih delova kôda (na konkretnom računaru) može da se proceni ukoliko je raspoloživa procena vremena izvršavanja pojedinih operacija
- Primer: na PC računaru sa procesorom Intel Core i7-2670QM na 2.2GHz koji radi pod Linux OS, za operaciju sabiranja dve vrednosti tipa `int` potrebno je $\sim 0,3ns$ a za operaciju množenja $\sim 1ns$
- Ova procenu treba uzimati sa rezervom, jer ona ne podrazumeva neke bitne aspekte izvršavanja programa (koji se još procesi trenutno izvršavaju, koji je operativni sistem, kako je napravljena izvršna verzija programa itd)

Profajleri

- Procenu vremena izvršavanja pojedinih operacija i kontrolnih struktura na računaru moguće je izmeriti
- Postoje mnogi alati za analizu i unapređenje performansi programa i nazivaju se **profajleri** (*profiler*), a njihova osnovna uloga je da daju podatke o tome koliko puta je koja funkcija pozvana tokom (nekog konkretnog) izvršavanja, koliko je utrošila sistemskog vremena itd.
- Ukoliko se razvijeni programski kôd ne izvršava dovoljno brzo, prvi kandidati za izmenu su delovi kôda koji troše najviše vremena
- Primeri Python profajlera: **cProfile**, **line_profiler**, **memory_profiler**, **Scalene**, **Pyinstrument**, **py-spy** itd.

Funkcija složenosti algoritma

- **Funkcija složenosti** algoritma je analitička funkcija koja povezuje veličinu problema (ulaznih podataka) i vremena izvršavanja algoritma ili utrošenih memorijskih reursa
- Vremenska i prostorna složenost mogu se razmatrati:
 - u okvirima konkretnog vremena ili prostora utrošenog za konkretne ulazne podatke
 - u okvirima asimptotskog ponašanja vremena ili prostora kada se količina ulaznih podataka povećava
- Vremenska i prostorna složenost algoritma određuje i njegovu praktičnu upotrebljivost tj. najveću dimenziju ulaza za koju je moguće da će se algoritam izvršiti u nekom razumnom vremenu ili koristeći postojeće resurse

Funkcija vremenske složenosti

- Za algoritme A i B porede se njihove funkcije složenosti $t_A(n)$ i $t_B(n)$ i porede se brzine rasta ove dve funkcije sa porastom ulaza n

Efikasniji je onaj algoritam čija funkcija složenosti sporije raste u opštem slučaju, za proizvoljno veliko n

- Za algoritme A i B važi:
$$t_B(n) > t_A(n) \text{ za } n > \frac{c-b}{a}.$$
- Univerzalni pristup u analizi funkcije složenosti koji neće zavisiti od hardverskih i softverskih karakteristika ciljnog sistema podrazumeva sledeća pojednostavljenja:
 - trajanje izvršavanja algoritma za određenu dimenziju problema treba izraziti preko broja dominantnih operacija u algoritmu
 - funkciju složenosti treba posmatrati u graničnom slučaju kada brojnost ulaznih podataka teži beskonačnosti

Analiza slučajeve funkcije složenosti

- Algoritmi u praksi imaju kompleksne ulazne podatke sa različitim raspodelama verovatnoća
- Zato se funkcije složenosti mogu posmatrati za 3 različita slučaja:
 - Najpovoljniji slučaj
 - Prosečan slučaj
 - Najnepovoljniji ili najgori slučaj!!!

```
def is_in(L, e):  
    for i in L:  
        if i == e:  
            return True  
    return False
```

Primer: proverava da li se karakter **e** nalazi u listi tako što se kroz nju prolazi element po element

- Kada je **e** na početku liste → **BEST CASE**
- Kada je **e** na sredini liste → **AVERAGE CASE**
- Kada **e** nije u listi → **WORST CASE**

Analiza slučajeva funkcije složenosti

- Najčešće se algoritmi ne izvršavaju isto za sve ulaze istih veličina
 - Analiza najgoreg slučaja vrši procenu složenosti algoritma za najgori slučaj (slučaj da se algoritam najduže izvršava ili troši najviše memorije) i daje jake garancije da program koji je u najgorem slučaju dovoljno efikasan, u svim slučajevima može da se izvrši sa raspoloživim resursima
 - Nekada je moguće izračunati prosečno vreme izvršavanja algoritma, ali je za to potrebno precizno poznavanje opsega ulaznih vrednosti i verovatnoće pojave tih vrednosti; Procena prosečnog izvršavanja može biti varljiva i previše optimistična
 - Analiza najboljeg slučaja je previše optimistična i nema smisla
- Nekada se analiza vrši tako što se procenjuje ukupno vreme potrebno da se izvrši određeni broj srodnih operacija, i to se naziva **amortizovana analiza** (bitno je zbirno vreme izvršavanja svih operacija)

Analiza algoritama

3. O - Θ - Ω notacije i klase složenosti algoritama

Asimptotska analiza algoritama

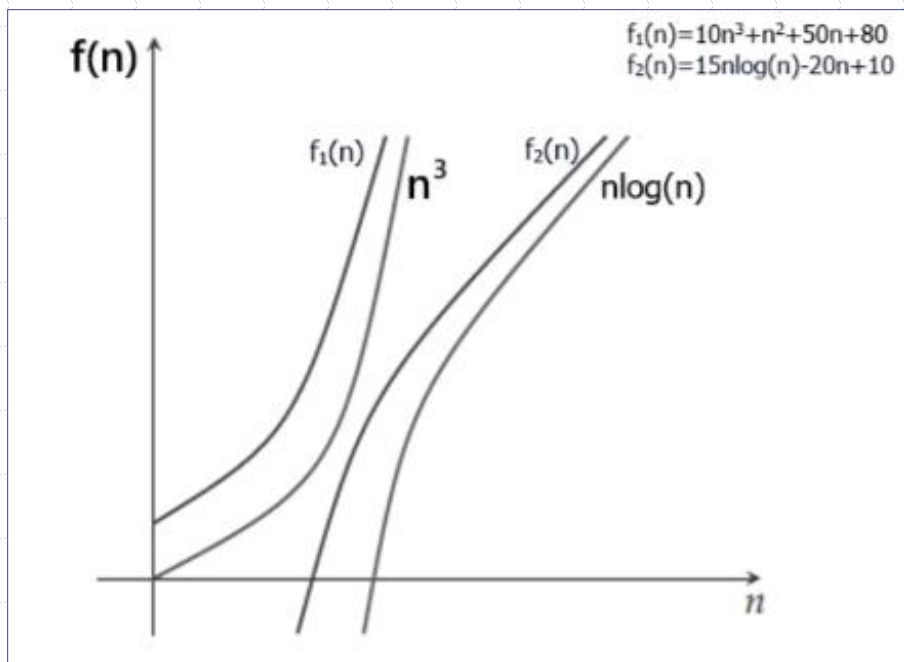
- Tačno vreme izvršavanja algoritama je unapred skoro nemoguće izračunati pa se vrši njihova procena ili evaluacija
- Usled kompleksnosti ili nemogućnosti izvođenja egzaktne formule za broj operacija nekog algoritma, koristi se asimptotsko ponašanje broja operacija kad veličina ulaznih podataka neograničeno raste
- Analizu algoritma pojednostavljujemo i tražimo red veličine funkcije vremena njegovog izvršavanja za veliki broj ulaznih elemenata
- Približno vreme izvršavanja naziva se asimptotsko vreme izvršavanja algoritma i predstavlja meru brzine rasta tačnog vremena izvršavanja algoritma sa povećanjem broja njegovih ulaznih podataka
- Za opis vremenske složenosti koristi se termin asimptotska notacija
- Član funkcije vremena izvršavanja sa najvećim eksponentom (vodeći član) najviše utiče na njen rast pa je on najvažniji za opis vremenske složenosti

Asimptotska analiza algoritama

- **Cilj:** Analiza povećanja vremena izvršavanja algoritma sa povećanjem veličine ulaza
- Metoda koja povezuje veličinu ulaza sa brojem operacija
- Zna se izraz za broj operacija potrebnih za izvršavanje algoritma, želi se znati asimptotsko ponašanje kako veličina problema raste
 - Utvrđivanje granica rasta
 - Ne koristi se precizan proračun: reč „red rasta“ a ne „tačan rast“
- Fokus je na članu koji najbrže raste
- Zanimaju se aditivne i multiplikativne konstante, jer se želi znati koliko brzo se vreme izvršavanja povećava sa povećanjem veličine ulaza
- Naziv: red rasta (*order of growth*)
- Koriste se matematički pojmovi „veliko O“ i „veliko TH“

Asimptotska analiza algoritama

- Pojednostavljenje proračuna vremena izvršavanja algoritma podrazumeva:
 - Vreme izvršavanja algoritama značajno se povećava sa povećanjem broja ulaznih podataka
 - Za dovoljno veliku vrednost n , funkcija vremena izvršavanja $f(n)$ je određena vodećim članom izraza funkcije
 - Jednostavna funkcija vremena izvršavanja uzima se bez konstanti, koje su u odnosu na vodeći član najčešće beznačajne



- Krive $f_1(n)$ i n^3 , kao i $f_2(n)$ i $n \log(n)$ su vrlo blizu za velike vrednosti n
- Za velike vrednosti n može se smatrati da su ove krive jednake
- Asimptotsko vreme izvršavanja algoritama može se napisati izrazima

$$f_1(n) \approx n^3 \text{ i } f_2(n) \approx n \log(n)$$

O-notacija

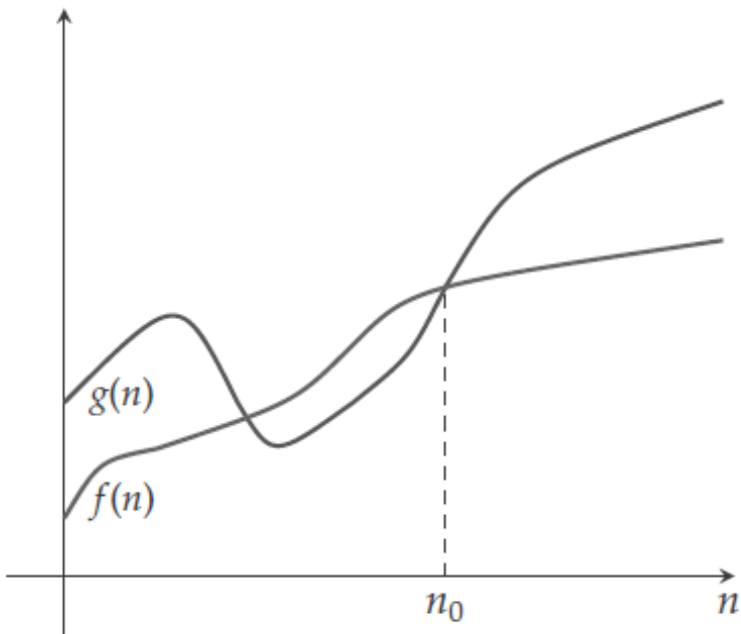
- O-notacija (Edmund Landau, Paul Bachman, početak XX veka) definiše procenu vremensko/prostornih resursa algoritama
- Matematička notacija „Big O“
- Predstavlja granično ponašanje funkcije kada argument teži nekoj specifičnoj vrednosti ili beskonačnosti
- Definiše se preko jednostavnijih funkcija
- Gornja granica složenosti algoritma se izražava O-notacijom:

Ako postoje pozitivna realna konstanta c i prirodan broj N takvi da za funkcije f i g nad prirodnim brojevima važi
 $f(n) \leq c \cdot g(n)$ za sve prirodne brojeve $n > N$ onda pišemo
 $f = O(g)$ i čitamo "f je veliko 'o' od g"

- "O" ne označava konkretnu funkciju, već klasu funkcija i uobičajeni zapis $f = O(g)$ zapravo znači $f \in O(g)$

O-notacija

- O-notacija se koristi za precizno defnisanje pojma da je neka funkcija „manja“ od druge
- Izrazom $f(n)=O(g(n))$ se opisuje svojstvo da je funkcija f manja od funkcije g , odnosno da g dominira nad f



- $f(n)=O(g(n))$ znači da postoji tačka n_0 takva da za sve vrednosti n od te tačke, funkcija $f(n)$ je ograničena vrednošću proizvoda pozitivne konstante c i funkcije $g(n)$
- Funkcija $g(n)$ je asimptotska gornja granica za funkciju $f(n)$
- Da bismo za konkretne funkcije pokazali da je vreme izvršavanja algoritma $T(n)=O(g(n))$ potrebno je naći n_0 i $c>0$ tako da je $T(n) \leq cg(n)$ za svako $n \geq n_0$

O-notacija

- Zanemariti aditivne i multiplikativne konstante
- Glavni su članovi funkcije najvišeg stepena

$$O(n^2) : n^2 + 2n + 2$$

$$O(n^2) : n^2 + 1000000n + 3^{1000}$$

$$O(n) : \log(n) + n + 4$$

$$O(n \log n) : 0.0001 * n * \log(n) + 300n$$

$$O(3^n) : 2n^{30} + 3^n$$

O-notacija

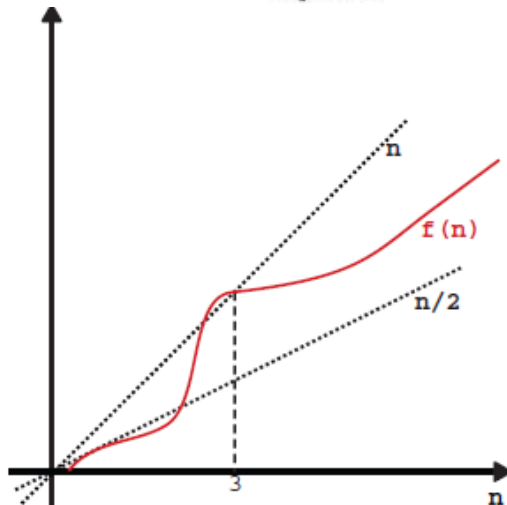
- Primer: U izrazu ($5n^2 + 1$) konstanta 1 je aditivna, a konstanta 5 je multiplikativna i ove konstante ne utiču na klasu kojoj funkcija pripada
- Zato se kaže da je neki algoritam složenosti $O(n^2)$ a ne $O(5n^2+1)$ jer aditivna konstanta 1 i multiplikativna 5 nisu od suštinske važnosti
- Ako jedan algoritam zahteva $5n^2+1$ instrukcija, a drugi n^2 , i ako se prvi algoritam izvršava na računaru koji je 6 puta brži od drugog, on će biti brže izvršen za svaku veličinu ulaza !!!
- Ako jedan algoritam zahteva n^2 instrukcija, a drugi n , ne postoji računar na kojem će prvi algoritam da se izvršava brže od drugog za svaku veličinu ulaza !!!

Klase složenosti

	logaritamska	linearna	linearitmička	kvadratna	kubna	exponenc.	faktoriel
n	$O(\log(n))$	$O(n)$	$O(n \log(n))$	$O(n^2)$	$O(n^3)$	$O(2^n)$	$O(n!)$
10	~ 3.32	10	~ 33	100	1000	1024	3628800
100	~ 6.64	100	~ 664	10000	1000000	$\sim 10^{30}$	$\sim 10^{156}$
1000	~ 10	1000	~ 9965	1000000	10^9	$\sim 10^{300}$	$> 10^{2567}$
1000000	~ 20	1000000	~ 19931568	10^{12}	10^{18}	$\sim 10^{3000000}$	$> 10^{5565700}$

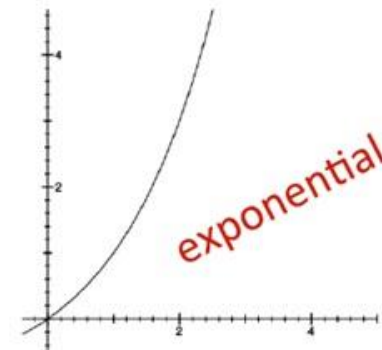
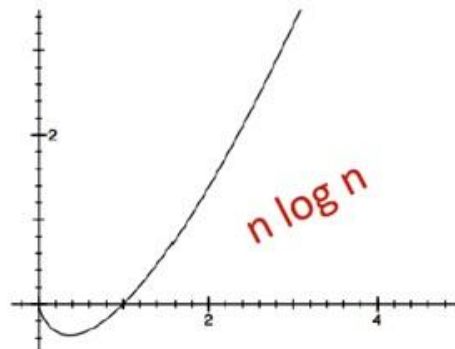
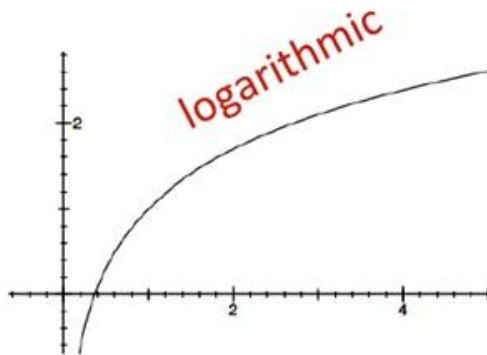
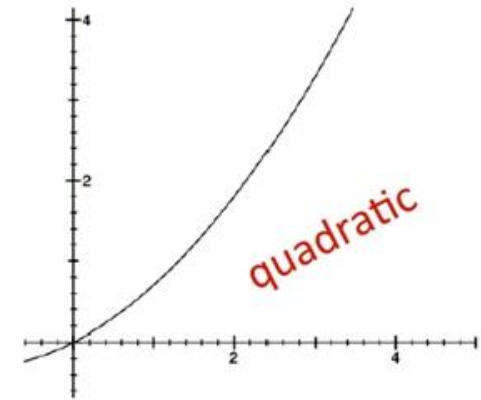
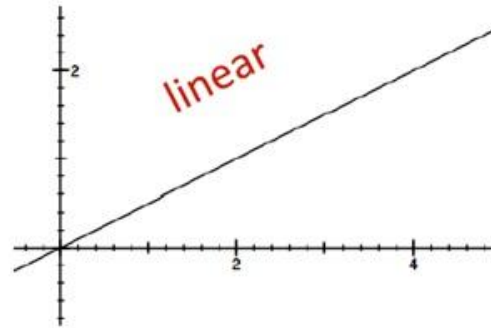
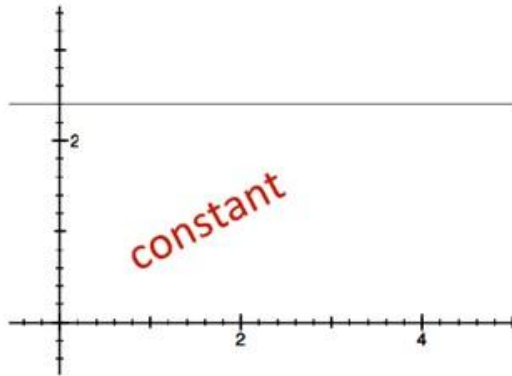
porast funkcije

sporo    brzo



O-notacija se koristi za precizno defnisanje pojma da je neka funkcija „manja“ od druge, ili da je u granicama između dve funkcije

Klase složenosti



Klase složenosti

$O(1)$ – konstantna složenost	Algoritmi koji se trenutno izvršavaju npr. koji izračunavaju matematičke funkcije, pristup elementu niza ili liste itd...
$O(n)$ – linearna složenost	Algoritmi kod kojih je potrebno proći kroz ceo niz, npr. održivanje max/min niza, učitavanje ili ispis svih elemenata niza ili liste
$O(n^2)$ – kvadratna složenost	Algoritmi sa ugnežđenim petljama, algoritmi za sortiranje selekcijom ili umetanjem
$O(n^3)$ – kubna složenost	Algoritmi sa višestruko ugnežđenim petljama, algoritmi za množenje matrica
$O(\log(n))$ – logaritamska složenost	Izuzetno efikasni, algoritmi binarne pretrage
$O(n\log(n))$ – log-linearna složenost	Efikan algoritam zasnovan na dekompoziciji (npr. sortiranje objedinjavanjem), korišćenju struktura podataka sa log vremnom pristupa, najpoznatiji Merge sort i Heap sort algoritmi
$O(2^n)$ – eksponencijalna složenost	Izuzetno neefikasni, algoritmi ispitivanja svih podskupova
$O(n!)$ – faktorijel složenost	Izuzetno neefikasni, algoritmi ispitivanja svih permutacija

Θ i Ω - notacije

- Θ i Ω notacije su takođe asimptotske notacije
- Kao što je O-notacija analogna relaciji $\leq$ za funkcije, definišu se analogni zapisi za relacije $\geq$ i $=$

Θ -notacija: Ako postoje pozitivne konstante c_1 i c_2 i prirodan broj N takvi da za funkcije f i g nad prirodnim brojevima važi $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ za sve brojeve $n > N$, onda važi $f(n) = \Theta(g(n))$ i čitamo „ f je veliko ‘teta’ od g “

- Θ -notacija pokazuje da funkcija $g(n)$ predstavlja asimptotski najbolju ocenu za funkciju $f(n)$

Ω -notacija: Ako postoje pozitivna realna konstanta c i prirodan broj N takvi da za funkcije f i g nad prirodnim brojevima važi $f(n) \geq c \cdot g(n)$ za sve prirodne brojeve $n > N$ onda je $f = \Omega(g)$ i čitamo „ f je veliko ,omega' od g ,“

- Ω -notacija pokazuje da funkcija $g(n)$ predstavlja asimptotski donju granicu za funkciju $f(n)$

Vreme izvršavanja i klase složenosti

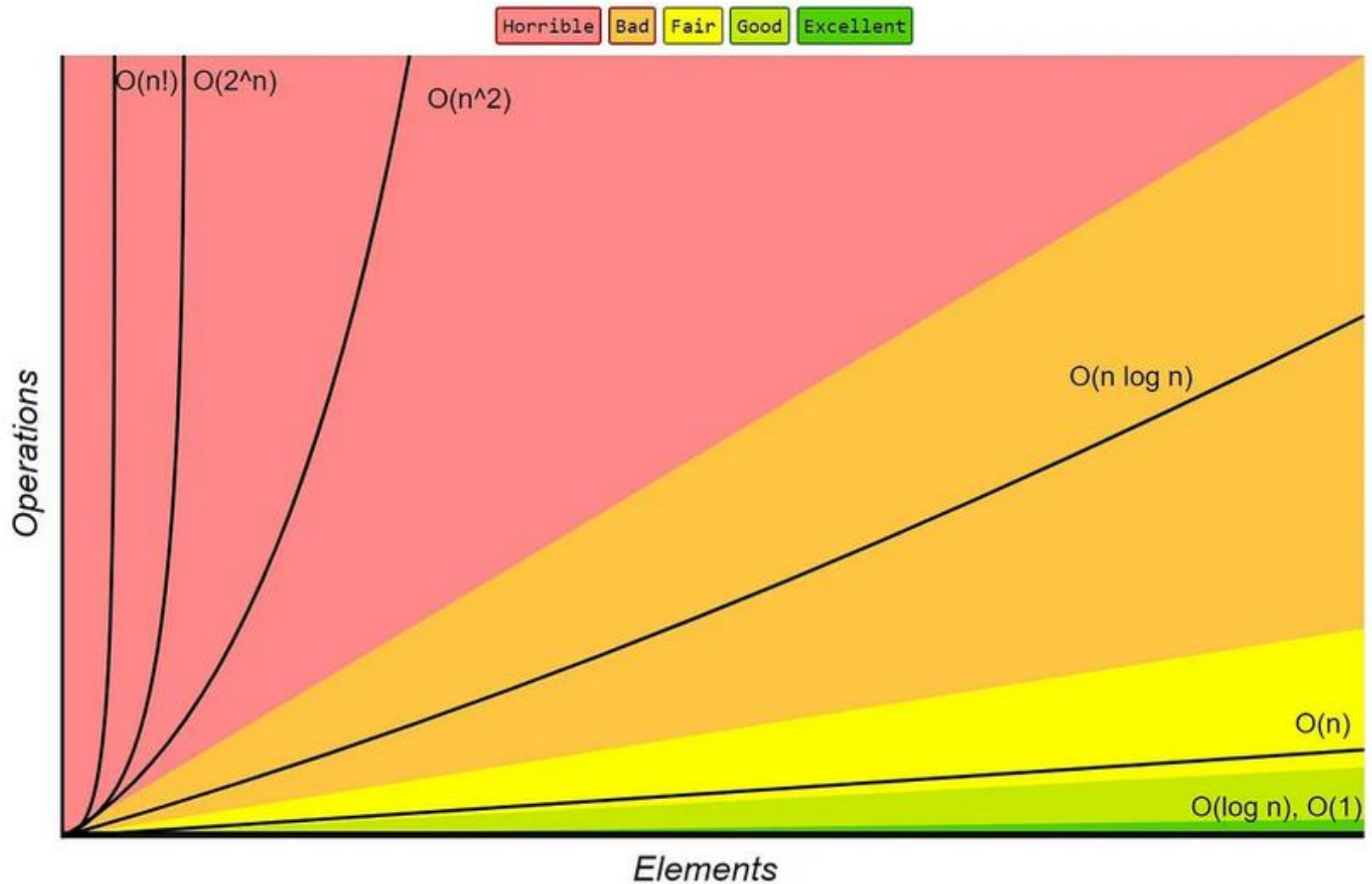
- Neka je funkcija $f(n)$ jednaka broju instrukcija koje zadati algoritam izvrši za ulaz veličine n (logaritamska, linearna, log-linearna, kvadratna, eksponencijalna i faktorijel)
- Tabela prikazuje potrebno vreme izvršavanja algoritma ako se pretpostavi da je za izvršavanje jedna instrukcije potrebno vreme od 1ns

$n \setminus f(n)$	$\log n$	n	$n \log n$	n^2	2^n	$n!$
10	$0.003 \mu s$	$0.01 \mu s$	$0.033 \mu s$	$0.1 \mu s$	$1 \mu s$	$3.63 ms$
20	$0.004 \mu s$	$0.02 \mu s$	$0.086 \mu s$	$0.4 \mu s$	$1 ms$	$77.1 god$
30	$0.005 \mu s$	$0.03 \mu s$	$0.147 \mu s$	$0.9 \mu s$	$1 s$	$8.4 \times 10^{15} god$
40	$0.005 \mu s$	$0.04 \mu s$	$0.213 \mu s$	$1.6 \mu s$	$18.3 min$	
50	$0.006 \mu s$	$0.05 \mu s$	$0.282 \mu s$	$2.5 \mu s$	$13 dan$	
100	$0.007 \mu s$	$0.1 \mu s$	$0.644 \mu s$	$10 \mu s$	$4 \times 10^{13} god$	
1,000	$0.010 \mu s$	$1.0 \mu s$	$9.966 \mu s$	$1 ms$		
10,000	$0.013 \mu s$	$10 \mu s$	$130 \mu s$	$100 ms$		
100,000	$0.017 \mu s$	$0.10 \mu s$	$1.67 ms$	$10 s$		
1,000,000	$0.020 \mu s$	$1 ms$	$19.93 ms$	$16.7 min$		
10,000,000	$0.023 \mu s$	$0.01 s$	$0.23 s$	$1.16 dan$		
100,000,000	$0.027 \mu s$	$0.10 s$	$2.66 s$	$115.7 dan$		
1,000,000,000	$0.030 \mu s$	$1 s$	$29.9 s$	$31.7 god$		

Vreme izvršavanja i klase složenosti

- Algoritmi čija je složenost **odozgo ograničena** polinom-funkcijama generalno su **efikasni**
- Algoritmi čija je složenost **odozdo ograničena** eksponencijalnom ili faktorijel funkcijom se smatraju **neefikasnim**
- Vodeći član funkcije $f(n)$ određuje potrebno vreme izvršavanja algoritma
- Primer iz tabele: Za broj instrukcija n^2+2n i ulaz veličine 1.000.000, član n^2 uzima **16.7 min.** dok član $2n$ uzima samo **2 μ s**
- Za $f(n)=2n^2+3n+2$ vreme od faktora $3n$ (3ms) i od faktora 2 (2ns) će biti zanemarljivo u odnosu na vreme od faktora $2n^2$ (~33 min)
- Vremenska (prostorna) složenost je uglavnom određena „vodećim” članom u izrazu koji određuje broj potrebnih instrukcija
- Na upotrebljivost algoritma ne utiču mnogo multiplikativne i aditivne konstante, koliko asimptotsko ponašanje broja instrukcija u zavisnosti od veličine ulaza

Klase složenosti



Klase složenosti struktura podataka

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
<u>Array</u>	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Stack</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Queue</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Singly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Doubly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Skip List</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n \log(n))$
<u>Hash Table</u>	N/A	$\theta(1)$	$\theta(1)$	$\theta(1)$	N/A	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Binary Search Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Cartesian Tree</u>	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	N/A	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>B-Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>Red-Black Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>Splay Tree</u>	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>AVL Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>KD Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$